WOXSEN UNIVERSITY
SCHOOL OF TECHNOLOGY

PROJECT-BASED LEARNING (PBL) REPORT

AIRLINE RESERVATION & FLIGHT OPERATIONS MANAGEMENT SYSTEM

Submitted in partial fulfillment of the requirements for

DATABASE MANAGEMENT SYSTEMS

Submission Detail	**Information**
Program	B.Tech AI & ML
Student Name	Nhaithik Reddy Siddenki
Roll Number	25WU0102185
Faculty	Dr. C Nagaraju
Academic Year	2026–2027

Submitted to the School of Technology, Woxsen University

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for providing the opportunity to complete this Project-Based Learning activity as part of the Database Management Systems course.

I thank Dr. C Nagaraju for the guidance and academic support provided during the development of the Airline Reservation & Flight Operations Management System. This project helped me apply database concepts including relational modelling, SQL operations, integrity constraints, views, triggers, database connectivity, and application development.

I also acknowledge the documentation and development tools used during the project. The implementation and report are intended to demonstrate practical learning; details that depend on the live database schema or tests are identified for final verification.

UNDERTAKING

I hereby declare that the project entitled “Airline Reservation & Flight Operations Management System” is submitted as part of the Project-Based Learning activity for the Database Management Systems course.

I confirm that the report describes the project work to the best of my knowledge and that external resources, software documentation, and tools used have been acknowledged where applicable. I understand that the final submission must accurately represent the implemented database, application behavior, screenshots, and test outcomes.

Detail	
Name	Nhaithik Reddy Siddenki
Roll Number	25WU0102185
Program	B.Tech AI & ML
Date	_____
Signature	_____

TABLE OF CONTENTS

Section	Page
Abstract	8
List of Tables	5
List of Figures	6
List of Acronyms	7
1. Problem Statement, Solution Strategy and ER Diagram	9
1.1 Problem Statement	9
1.2 Objectives	9
1.3 Scope and Assumptions	9
1.4 Proposed Solution Strategy	10
1.5 Functional Requirements	10
1.6 Entity-Relationship Diagram (ERD)	11
1.7 Entity and Relationship Summary	12
2. Logical Database Design	13
2.1 Relational Schema	13
2.2 Keys and Integrity Constraints	13
2.3 Functional Dependencies	14
2.4 Normalization up to Third Normal Form	14
2.5 Data Dictionary	14
2.6 Implementation of Logical Design: DDL and Sample Data	16
3. Application Development Using Logical Database Design	18
3.1 Application Overview	18
3.2 Development Environment and Database Connectivity	18
3.3 Application Modules and Database Mapping	18
3.4 CRUD Operations	19
3.5 Business Rules, Validation and Error Handling	19
3.6 Reports and Database Operations	20
3.7 Application Screens	20
3.8 Testing and Results	23
4. Conclusion	26
4.1 Limitations and Future Enhancements	26
5. References	27
6. Appendices	28

LIST OF TABLES

Table No.	Table Title
Table 1	Scope and Assumptions
Table 2	Proposed Solution Strategy
Table 3	Functional Requirements
Table 4	Entity and Relationship Summary
Table 5	Relational Schema
Table 6	Keys and Integrity Constraints
Table 7	Normalization Status up to 3NF
Table 8	Table-Level Data Dictionary
Table 9	Database Implementation Environment
Table 10	Application Development Environment
Table 11	Application Module-to-Database Mapping
Table 12	CRUD Operations
Table 13	Business Rules, Validation and Error Handling
Table 14	Reports and Database Operations
Table 15	Test Cases

LIST OF FIGURES

Figure No.	Figure Title
Figure 1	Conceptual ER relationship overview (verify against live schema before submission)
Figure 2	Application dashboard — live screenshot to be inserted
Figure 3	Flight Search & Booking screen — live screenshot to be inserted
Figure 4	Operations Center screen — live screenshot to be inserted
Figure 5	Reports screen — live screenshot to be inserted

LIST OF ACRONYMS

Acronym	Expansion
CRUD	Create, Read, Update, Delete
DBMS	Database Management System
DDL	Data Definition Language
DML	Data Manipulation Language
ERD	Entity-Relationship Diagram
FK	Foreign Key
GUI	Graphical User Interface
PK	Primary Key
SQL	Structured Query Language
3NF	Third Normal Form
UI	User Interface

ABSTRACT

Airline reservation and flight operations require coordinated handling of flight schedules, passengers, reservations, seats, tickets, check-in, baggage, payments, cancellations, and reporting. When these activities are managed through disconnected records, it becomes difficult to maintain consistent data, enforce booking rules, and obtain reliable operational summaries. The objective of this project is to develop a database-driven Airline Reservation & Flight Operations Management System using Python Flask, MySQL, HTML, CSS, and JavaScript. The project reuses an existing MySQL database named airline_reservation and connects it to a Flask web application. The database includes operational tables for flights, passengers, reservations, seats, tickets, check-ins, baggage, payments, cancellations, airports, routes, aircraft, and fares, along with reported views and triggers. The application provides a dashboard, data-management screens, a flight-search and booking workflow, an operations center, and reports. The implemented interface has been observed displaying a successful MySQL connection and dashboard summary values during development. Detailed schema attributes and end-to-end test outcomes must be confirmed against the live database and recorded screenshots before final submission. The project demonstrates how relational database concepts can support a connected, maintainable airline operations application.

Keywords: Airline Reservation, MySQL, Flask, Relational Database, SQL Triggers, CRUD Operations

CHAPTER 1
PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

An airline reservation process involves related records for flights, routes, airports, aircraft, passengers, reservations, seat assignments, tickets, check-ins, baggage, payments, and cancellations. If each activity is maintained independently, the same information may be entered repeatedly and operational updates can become inconsistent. A booking must be associated with a valid passenger and flight; payment and ticketing must correspond to the appropriate reservation; and check-in, baggage, cancellation, and refund actions must follow defined business rules.

This project addresses the need for a single database-backed application that presents these operations through a consistent web interface. It uses a MySQL relational database and a Python Flask backend to query and update existing records. The project scope is an academic prototype; it is not represented as a production airline reservation service or a payment gateway.

1.2 Objectives

- Connect a Python Flask application to the existing airline_reservation MySQL database.
- Provide dashboard and management screens for the available operational entities.
- Support flight search and a reservation workflow using existing passenger and flight records.
- Apply validation and database business rules to booking, payment, ticketing, check-in, baggage, and cancellation operations where implemented.
- Present operational summaries and reports using database queries and views.
- Document the schema, application mapping, and test evidence in a consistent DBMS PBL report.

1.3 Scope and Assumptions

Item	Project-specific description
In scope	Flask web application; MySQL connectivity; flight, passenger, reservation, seat, ticket, check-in, baggage, payment, cancellation, airport, route, aircraft and fare screens; booking and operations workflow; reports.
Out of scope	Live airline/GDS integration, real payment processing,

	production-grade identity management, real-world ticket issuance, external airport feeds, and deployment availability guarantees.
Assumptions	The MySQL server is accessible from the machine running Flask; the database airline_reservation already exists; required tables, sample data, views and triggers are maintained by the database owner; local credentials are stored in .env.
Data integrity	Existing triggers and constraints are treated as authoritative. SQL scripts should be inspected before execution; the database must not be dropped or recreated as part of normal startup.

1.4 Proposed Solution Strategy

The system uses a layered approach. HTML templates and CSS provide the interface; JavaScript handles client-side interactions; Flask routes validate requests and execute database operations; and MySQL stores persistent records and enforces database-level constraints. A dashboard gives an overview of selected counts and totals. Generic module pages provide table-oriented data management, while the booking and operations pages support the reservation lifecycle.

Solution element	Project-specific description
Primary users	Student demonstrator or airline operations administrator in the academic prototype.
Major modules	Dashboard, flight search and booking, operations center, flights, passengers, reservations, seats, tickets, check-in, baggage, payments, cancellations, airports, routes, aircraft, fares, and reports.
Proposed workflow	Flight Search → Booking → Seat Allocation → Payment → Ticket Issue → Check-In → Baggage → Cancellation/Refund → Reports.
Business rules	Validate related records and reservation status; use existing database triggers; avoid double-adjusting available seat inventory; require prerequisites for downstream operations.
Expected outputs	Dashboard summaries, flight and reservation lists, revenue and route-demand summaries, occupancy and passenger-manifest reports where available.

1.5 Functional Requirements

Req. ID	Functional requirement	Priority
FR-01	The application shall connect to the existing airline_reservation database.	High
FR-02	The dashboard shall display selected operational counts and connection status.	High
FR-03	The user shall be able to search available flights using route/date criteria supported by the application.	High
FR-04	The application shall create a reservation associated with an existing	High

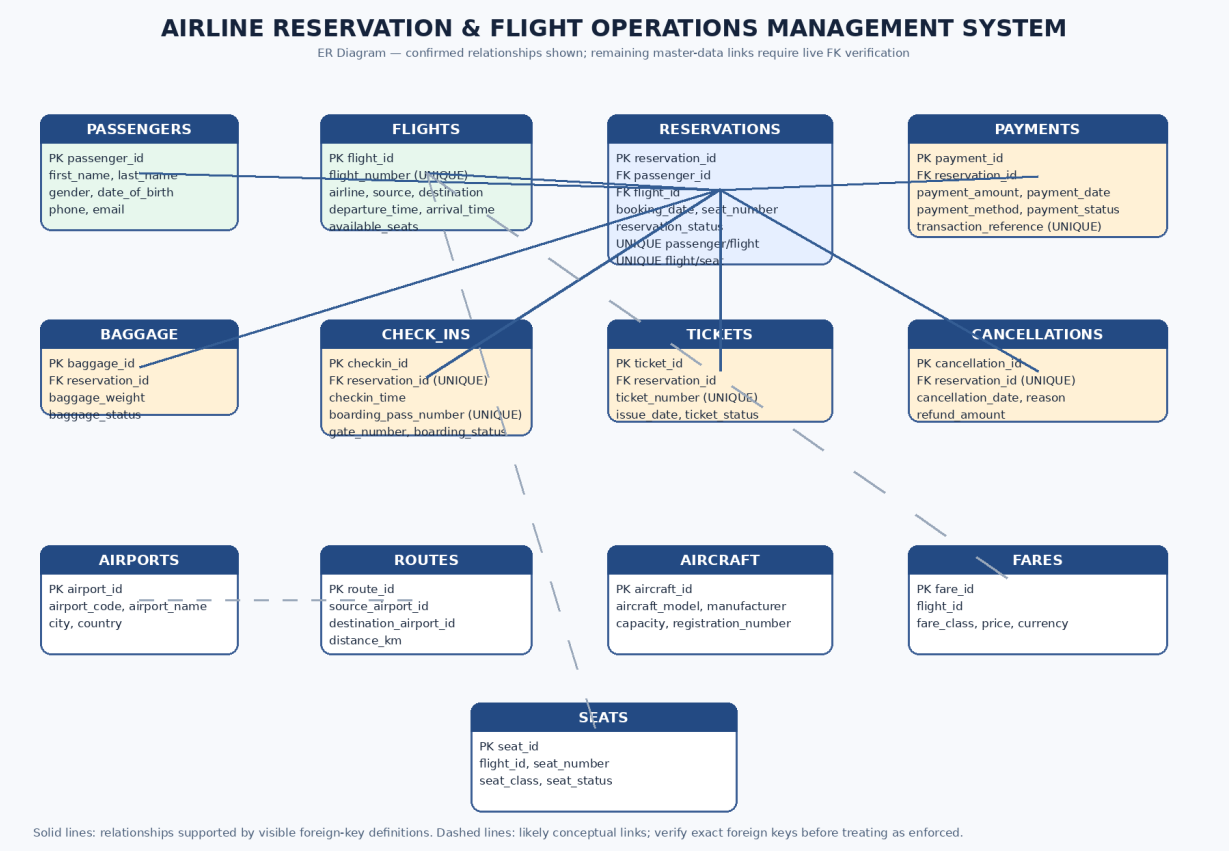
	passenger and flight.	
FR-05	The operations workflow shall provide payment, ticket, check-in, baggage, and cancellation actions subject to validation.	High
FR-06	Management pages shall display records for supported database modules.	High
FR-07	The application shall provide database-backed reports for operational summaries.	Medium
FR-08	Database credentials shall be configurable locally and excluded from source control.	High

1.6 Entity-Relationship Diagram (ERD)

The implemented database contains the following named tables: aircraft, airports, baggage, cancellations, check_ins, fares, flights, passengers, payments, reservations, routes, seats, and tickets. The relationships below are a conceptual guide inferred from the operational roles of these tables; exact foreign-key columns and cardinalities must be confirmed by inspecting the live schema before this figure is treated as the approved ERD.

Entity	Likely relationship to review in live schema
airports	Airport records are associated with route origins/destinations and/or flight locations.
routes	Routes relate airport pairs and are associated with flights.
aircraft	Aircraft are assigned to flights; capacity and seat allocation should be consistent.
flights	Flights are linked to routes/aircraft and have reservations and available-seat information.
passengers	Passengers can have reservation records.
reservations	Reservation is the central transaction entity associated with a passenger and flight; related to payment, ticket, check-in, baggage, and cancellation records.
seats	Seat records/assignments relate to a flight and/or reservation.
payments	Payment records are associated with reservations.
tickets	Ticket records are associated with reservations.
check_ins	Check-in records are associated with reservations/tickets.
baggage	Baggage records are associated with a reservation or check-in.
cancellations	Cancellation and refund records are associated with reservations.
fares	Fare information is associated with flights/routes according to the implemented schema.

Figure 1. ER diagram of the Airline Reservation & Flight Operations Management System. Solid lines indicate relationships supported by the visible database definitions; dashed lines are conceptual and require exact foreign-key verification.



1.7 Entity and Relationship Summary

Entity / relationship	Purpose	Verification note
Flight / route / airport	Represent flight movement and origin/destination information.	Confirm actual FK attributes in live schema.
Aircraft / seats	Represent aircraft and seat capacity/assignment.	Confirm whether seats are static inventory or reservation assignments.
Passenger / reservation	Associate passenger records with bookings.	Confirm reservation FK and status domain.
Reservation / payment / ticket	Support payment and ticketing lifecycle.	Confirm uniqueness and payment-state constraints.
Reservation / check-in / baggage	Track post-booking operations.	Confirm prerequisites and relationship cardinality.
Reservation / cancellation	Track cancellation and refund details.	Existing cancellation triggers were reported in the project context.

CHAPTER 2
LOGICAL DATABASE DESIGN

2.1 Relational Schema

The application uses the existing MySQL database airline_reservation. The known table list was observed during project development and is listed in Table 5. Exact column names, data types, key definitions, and referential actions should be taken from DESCRIBE and SHOW CREATE TABLE output for the final database, rather than inferred from table names.

S.No.	Table name	Purpose
1	aircraft	Aircraft information
2	airports	Airport reference information
3	baggage	Baggage records
4	cancellations	Cancellation and refund records
5	check_ins	Check-in records
6	fares	Fare information
7	flights	Flight records and seat availability
8	passengers	Passenger records
9	payments	Payment transactions
10	reservations	Reservation/booking records
11	routes	Route information
12	seats	Seat records or assignments
13	tickets	Ticket records

2.2 Keys and Integrity Constraints

The application discovers table columns and primary-key metadata at runtime for its generic module screens. The live schema also includes triggers used for reservation and operations rules. Because the complete SHOW CREATE TABLE output was not included with the report inputs, the exact PK/FK/UNIQUE/NOT NULL/CHECK/DEFAULT definitions are not reproduced here as unverified facts.

Constraint type	Role in the database	Project-specific verification
PRIMARY KEY	Uniquely identifies a row in each relation.	Inspect SHOW CREATE TABLE for all 13 tables.
FOREIGN KEY	Maintains valid relationships between related records.	List each FK and its referenced table/column from information_schema.
UNIQUE	Prevents duplicate values or duplicate business keys where defined.	Verify constraints for ticket numbers, seat assignments, and transaction references.
NOT NULL / DEFAULT	Requires essential values and provides defaults where configured.	Read exact column definitions from the live schema.
CHECK	Restricts values to an allowed domain where supported.	Verify status, amounts, and other check constraints.
Triggers	Enforce procedural rules during insert/update events.	Project context reports triggers for cancellations, check-in, and reservation seat-count updates.

2.3 Functional Dependencies

Functional dependencies must be derived from the actual candidate keys and business rules. At a conceptual level, a primary key functionally determines the non-key attributes of its own row. The exact dependencies cannot be stated attribute-by-attribute without the live table definitions. The following examples describe the intended pattern, not a substitute for schema verification:

- flight primary key → the non-key attributes stored for that flight.
- passenger primary key → the non-key attributes stored for that passenger.
- reservation primary key → the non-key attributes stored for that reservation.
- payment primary key → the non-key attributes stored for that payment.

2.4 Normalization up to Third Normal Form (3NF)

Normalization should be assessed from the implemented attributes and functional dependencies. The report cannot certify every table as 3NF without its complete column-level schema. The general design target is: 1NF requires atomic values and no repeating groups; 2NF requires non-key attributes to depend on the whole candidate key; 3NF requires non-key attributes not to depend transitively on candidate keys.

Relation group	1NF	2NF	3NF	Evidence required
Master/reference tables (airports, aircraft, routes)	Review	Review	Review	SHOW CREATE TABLE and dependency analysis
Operational tables (flights, passengers, reservations, seats)	Review	Review	Review	Column definitions, keys, and dependencies
Transaction tables (payments, tickets, check_ins, baggage, cancellations)	Review	Review	Review	Column definitions and transaction relationships
fares	Review	Review	Review	Fare key and applicable flight/route dependency

Normalization status: the schema-level assessment must be finalized from all live table definitions and candidate keys. This report does not certify every table as 3NF without the full verified column-level schema.

2.5 Data Dictionary

Table	Known purpose	Attribute-level dictionary status
aircraft	Aircraft reference data	Exact columns/types should

		be confirmed using DESCRIBE aircraft for exact columns/types.
airports	Airport reference data	Exact columns/types should be confirmed using DESCRIBE airports for exact columns/types.
baggage	Baggage handling records	Exact columns/types should be confirmed using DESCRIBE baggage for exact columns/types.
cancellations	Cancellation/refund records	Exact columns/types should be confirmed using DESCRIBE cancellations for exact columns/types.
check_ins	Check-in records	Exact columns/types should be confirmed using DESCRIBE check_ins for exact columns/types.
fares	Fare data	Exact columns/types should be confirmed using DESCRIBE fares for exact columns/types.
flights	Flight data and availability	Exact columns/types should be confirmed using DESCRIBE flights for exact columns/types.
passengers	Passenger records	Exact columns/types should be confirmed using DESCRIBE passengers for exact columns/types.

payments	Payment records	Exact columns/types should be confirmed using DESCRIBE payments for exact columns/types.
reservations	Reservation records	Exact columns/types should be confirmed using DESCRIBE reservations for exact columns/types.
routes	Route reference data	Exact columns/types should be confirmed using DESCRIBE routes for exact columns/types.
seats	Seat records/assignments	Exact columns/types should be confirmed using DESCRIBE seats for exact columns/types.
tickets	Ticket records	Exact columns/types should be confirmed using DESCRIBE tickets for exact columns/types.

2.6 Implementation of Logical Design: DDL and Sample Data

Item	Details
DBMS	MySQL
Database name	airline_reservation
Number of known tables	13
Known views	baggage_report_view, cancellation_report_view, flight_manifest_view, flight_occupancy_view, reservation_details, revenue_report_view, route_demand_view
Application access	Python Flask using the database connector configured by the project
DDL strategy	Existing database objects are reused; application startup is not intended to drop or recreate tables.

Representative schema verification commands are included below. Run them against the same database used by the application and attach the output in Appendix A/C if the course evaluator requires full DDL and a data dictionary.

```
USE airline_reservation;
SHOW TABLES;
DESCRIBE flights;
DESCRIBE reservations;
DESCRIBE passengers;
DESCRIBE payments;
DESCRIBE cancellations;
DESCRIBE check_ins;
DESCRIBE baggage;
SHOW FULL TABLES WHERE Table_type = 'VIEW';
SHOW TRIGGERS;
```

The exact DDL and complete column-level data dictionary should be exported from the live database for the appendix. The table and view names below are based on the supplied MySQL output; unprovided column definitions are not invented.

CHAPTER 3
APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

The application is a browser-based management interface implemented with Python Flask, HTML templates, CSS, and JavaScript. The Flask backend opens a database connection, retrieves rows and metadata, serves dashboard statistics, and provides routes for data-management and operational actions. The application reuses the existing MySQL database instead of recreating the schema.

3.2 Development Environment and Database Connectivity

Item	Details
Programming language	Python
Web framework	Flask
Database	MySQL; database name airline_reservation
Frontend	HTML, CSS, JavaScript
IDE / operating environment	Visual Studio Code on macOS, as observed in the development workflow
Configuration	Local .env file; .env.example documents expected configuration
Repository	https://github.com/Nhaithik/airline-reservation-flight-operations
Database connection evidence	The displayed dashboard showed “MySQL Connected” and “Connected to airline_reservation”.

The project’s connection settings are stored locally. The .env file must remain private and must not be committed to the public repository. The application should be run from the project directory with the environment activated and dependencies installed.

3.3 Application Modules and Database Mapping

Module / screen	Purpose	Tables / operations
Dashboard	Shows selected counts, totals, and database connection status.	Reads flights, passengers, reservations, payments and cancellations as applicable.
Flight Search & Booking	Searches flights and creates a reservation.	flights, passengers, reservations; availability and duplicate checks.
Operations Center	Supports reservation operations after booking.	reservations, payments, tickets, check_ins, baggage, cancellations; dependent workflow actions.
Flights	View/manage flight records.	flights; CRUD where enabled.
Passengers	View/manage passenger records.	passengers; CRUD where enabled.
Reservations	View reservation records.	reservations; CRUD may be restricted for workflow tables.

Seats	View seat information or allocation records.	seats and/or flight availability, as implemented.
Tickets	View issued ticket records.	tickets.
Check-In	View check-in records.	check_ins.
Baggage	View baggage records.	baggage.
Payments	View payment records.	payments.
Cancellations	View cancellation/refund records.	cancellations.
Airports, Routes, Aircraft, Fares	Manage reference/master data.	airports, routes, aircraft, fares.
Reports	Display operational summaries.	Known views include revenue_report_view, route_demand_view, flight_occupancy_view, flight_manifest_view, baggage_report_view, cancellation_report_view.

3.4 CRUD Operations

Operation	Application feature	Tables affected	Evidence
Create	Add a supported master or operational record; create booking.	Applicable module table; reservations for booking.	Confirm using a saved record and database query.
Read / Search	Dashboard, module lists, flight search, reports.	Tables/views used by the screen.	Dashboard and module screens were observed.
Update	Edit supported module records or progress a reservation workflow.	Table depends on the selected module/action.	Capture before/after values for submission.
Delete / controlled removal	Generic delete operations may be available for supported tables; workflow records may be restricted.	Table depends on module and configuration.	Verify a non-critical test record and any confirmation/constraint behavior.

3.5 Business Rules, Validation and Error Handling

Project notes and application code indicate workflow-specific validation. The exact observed response for each negative case should be captured during testing.

Rule / validation	Implementation purpose	Evidence status
Passenger and flight validation	A reservation should refer to existing passenger and flight records.	Code includes booking route validation; capture test result.
Seat availability / duplicate allocation	Prevent booking beyond availability and avoid duplicate active seat use.	Booking code includes seat/availability checks; verify seat inventory behavior against triggers.
Payment prerequisites	Restrict payment to eligible reservation states.	Workflow action implemented; run a positive and negative test.
Ticket issue prerequisites	Issue tickets only for reservations meeting workflow conditions.	Workflow action implemented; capture result.
Check-in prerequisites	Permit check-in only for eligible reservations/tickets; project context reports a check-in trigger for Confirmed reservations.	Trigger presence reported; verify exact rule with SHOW CREATE TRIGGER.
Cancellation after check-in	Project context reports a trigger preventing cancellation after check-in.	Verify using a test record or documented trigger output.
Refund amount validation	Project context reports a cancellation trigger validating refund amount against payment.	Verify the exact implemented rule before quoting numeric policy.
Database exceptions	Backend handles database errors and returns operation results to the UI.	Capture actual messages during test execution.
Credentials	Store real DB credentials locally in .env and exclude from Git.	Repository .gitignore and .env.example are present in the project bundle.

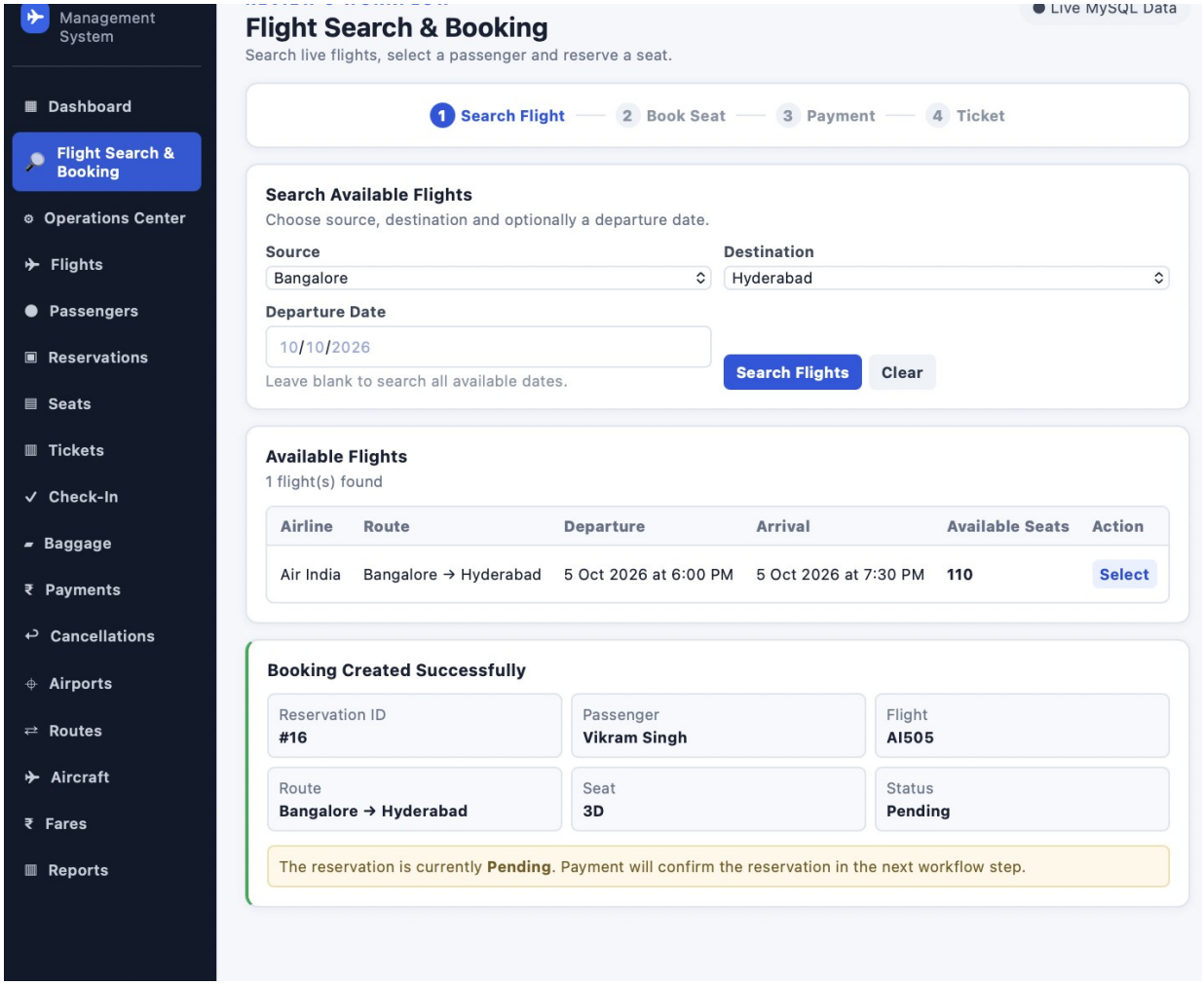
3.6 Reports and Database Operations

Report / operation	Database concept	Output / note
Revenue report	Aggregation over payment data and/or revenue view.	Dashboard previously displayed total revenue ₹34,300.00; report output should be captured.
Route demand	Grouping/aggregation by route.	Known view: route_demand_view.
Flight occupancy	Aggregation of booked/available capacity.	Known view: flight_occupancy_view; verify formula/status handling.
Passenger manifest	Join between flights and reservation/passenger/ticket data.	Known view: flight_manifest_view.
Baggage report	Baggage records summarized for operations.	Known view: baggage_report_view.
Cancellation report	Cancellation/refund records summarized.	Known view: cancellation_report_view.
Reservation details	Join/view of reservation-related data.	Known view: reservation_details.

3.7 Application Screens

The development session included a dashboard screenshot showing the title “Airline Reservation & Flight Operations”, a green MySQL Connected indicator, module navigation, and dashboard values. The screenshot showed 5 flights, 16 passengers, 11 reservations, 610 available seats, ₹34,300.00 total revenue, and 2 cancellations at the time it was captured. These are a point-in-time display, not guaranteed current totals.

Figure 2. Dashboard showing the MySQL connection indicator and summary cards.



The application screenshots below document the principal interface areas. The workflow evidence supplied for reservation #17 shows a completed sequence through payment, ticket issue, check-in and baggage entry. Screenshots represent the application state at the time of capture.

Figure 3. Flight Search & Booking interface showing search criteria and available flight results.

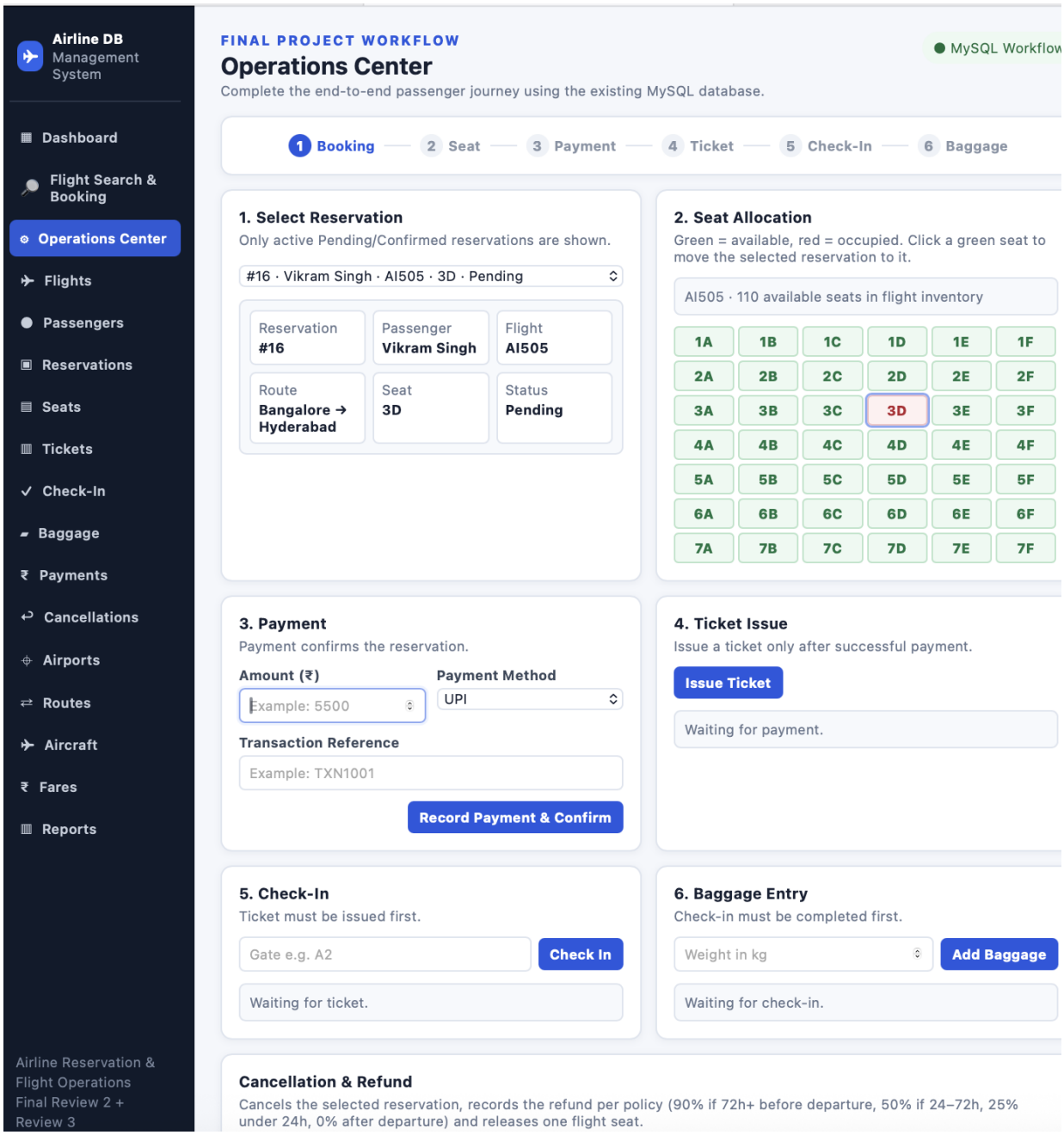


Figure 4. Operations Center interface for seat allocation, payment, ticket issue, check-in, baggage and cancellation.

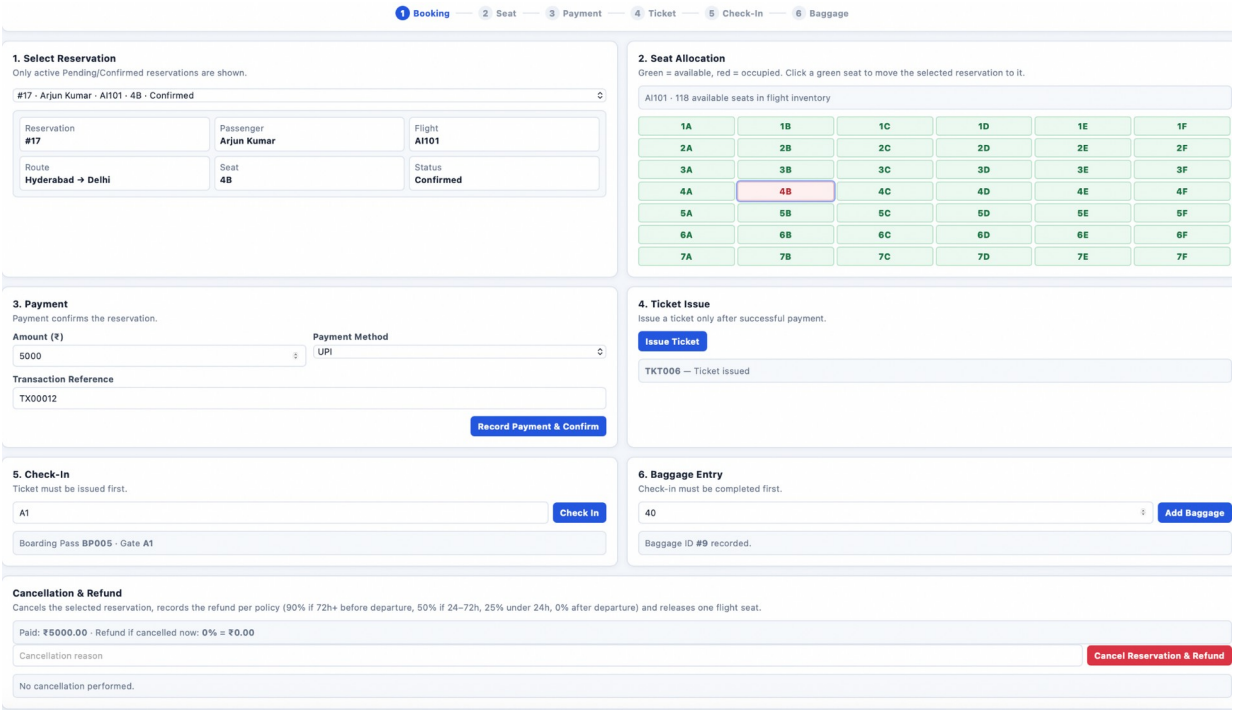
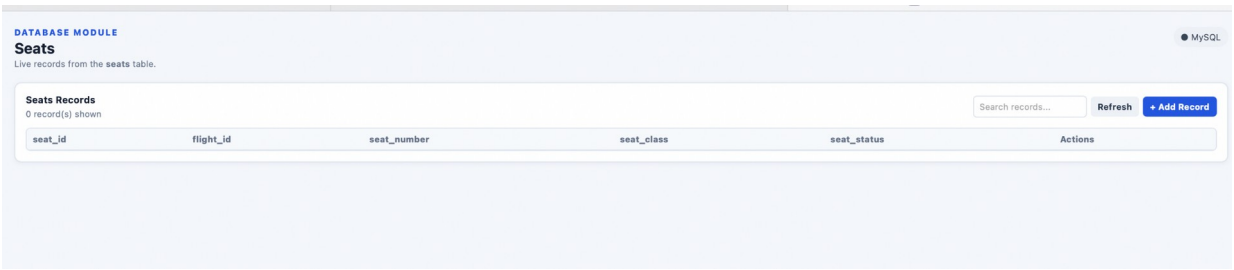


Figure 5. Database module/report evidence captured during development.



3.8 Testing and Results

Testing results are recorded only where supported by captured application or database output. The dashboard connection and the workflow for reservation #17 were observed. Negative validation cases and independent report reconciliation were not evidenced in the supplied screenshots and are therefore marked not verified or partial rather than passed.

Test ID	Feature	Action / evidence	Actual result	Status
TC-01	Database connectivity	Dashboard displayed MySQL Connected for airline_reservati	Connection indicator observed in application	PASS (observed)

		on.	screenshot.	
TC-02	Flight search	Flight Search & Booking displayed available flight results.	Five flights were displayed in the captured search screen.	PASS (observed)
TC-03	Reservation and seat	Reservation #17 shown for Arjun Kumar, flight AI101, seat 4B.	Reservation status displayed as Confirmed.	PASS (observed)
TC-04	Payment	Payment record for reservation #17.	₹5,000.00; UPI; status Paid; transaction reference TX00012 in the captured result.	PASS (observed)
TC-05	Ticket issue	Ticket issue step for reservation #17.	Ticket TKT006 shown as Issued.	PASS (observed)
TC-06	Check-in	Check-in step for reservation #17.	Boarding pass BP005 and gate A1 shown.	PASS (observed)
TC-07	Baggage	Baggage entry for reservation #17.	Baggage ID #9, weight 40.00 kg, status Checked-In shown.	PASS (observed)
TC-08	Negative validation	Invalid payment/seat/check-in/cancellation cases.	No complete captured negative-test evidence was	NOT VERIFIED

			supplied.	
TC-09	Reports	Open and compare report values with independent SQL output.	Report screenshot exists, but independent cross-check was not supplied.	PARTIAL

Observed result summary: the Flask application displayed a connection to airline_reservation. A captured end-to-end workflow for reservation #17 showed Confirmed status, a Paid payment of ₹5,000.00, ticket TKT006, boarding pass BP005 at gate A1, and baggage ID #9 recorded at 40.00 kg. Negative cases and independent reconciliation still require separate evidence.

Verified workflow evidence

The captured Operations Center result for reservation #17 records the following: reservation Confirmed; payment ₹5,000.00 marked Paid; ticket TKT006 marked Issued; boarding pass BP005 at gate A1; baggage ID #9 with 40.00 kg and status Checked-In. This is evidence of the displayed application state, not evidence of a live external payment transaction.

CHAPTER 4
CONCLUSION

4. Conclusion

The Airline Reservation & Flight Operations Management System demonstrates the integration of a relational MySQL database with a Python Flask web application. It brings multiple airline-operation areas into one interface, including flight and passenger management, reservations, seat allocation, tickets, check-in, baggage, payments, cancellations, and reports. The project uses an existing database rather than rebuilding it at application startup and provides a practical setting for applying DBMS concepts such as relational tables, SQL queries, views, constraints, triggers, and database connectivity.

During development, the dashboard displayed a successful connection to airline_reservation and showed summary counts and revenue. The report deliberately distinguishes this observed evidence from unrecorded workflow tests. Final submission quality depends on verifying the live schema, inserting the approved ER diagram, adding application screenshots, and recording actual expected-versus-actual test results.

4.1 Limitations and Future Enhancements

- Generate and attach the full column-level data dictionary, exact CREATE TABLE statements, foreign-key list, and trigger definitions from the live MySQL database.
- Complete and document positive and negative tests for booking, seat inventory, payment, ticket issue, check-in, baggage, cancellation, and refunds.
- Add authentication, role-based access control, stronger audit logging, and production-grade security controls if the application is extended beyond academic demonstration.
- Deploy Flask and MySQL to appropriately secured hosting if a public URL is required; GitHub repository hosting alone does not run the backend.
- Improve automated testing and add controlled test fixtures so database state can be checked repeatably.

CHAPTER 5

REFERENCES

[1] Oracle, “MySQL 8.4 Reference Manual,” Oracle. Available: https://dev.mysql.com/doc/ . Accessed: Oct. 10, 2026.

[2] Pallets Projects, “Flask Documentation.” Available: https://flask.palletsprojects.com/ . Accessed: Oct. 10, 2026.

[3] Python Software Foundation, “Python 3 Documentation.” Available: https://docs.python.org/3/ . Accessed: Oct. 10, 2026.

[4] GitHub, “GitHub Docs.” Available: https://docs.github.com/ . Accessed: Oct. 10, 2026.

CHAPTER 6
APPENDICES

APPENDIX A — DATABASE SCRIPT AND SCHEMA EVIDENCE

The application repository contains schema_check.sql, which selects airline_reservation, lists tables, describes key operational tables, and lists views and triggers. The complete DDL/DML of the pre-existing database was not included in the available project ZIP; therefore, this report does not claim that schema_check.sql is the complete database creation script.

```
USE airline_reservation;
SHOW TABLES;
DESCRIBE flights;
DESCRIBE reservations;
DESCRIBE passengers;
DESCRIBE payments;
DESCRIBE cancellations;
DESCRIBE check_ins;
DESCRIBE baggage;
SHOW FULL TABLES WHERE Table_type = 'VIEW';
SHOW TRIGGERS;
```

APPENDIX B — APPLICATION SOURCE CODE

GitHub repository: https://github.com/Nhaithik/airline-reservation-flight-operations

The repository includes app.py, requirements.txt, templates/, static/, .env.example, .gitignore, README.md, DATABASE_NOTES.txt, schema_check.sql, and tests/ in the local project version. Keep .env private and do not publish passwords or API keys.

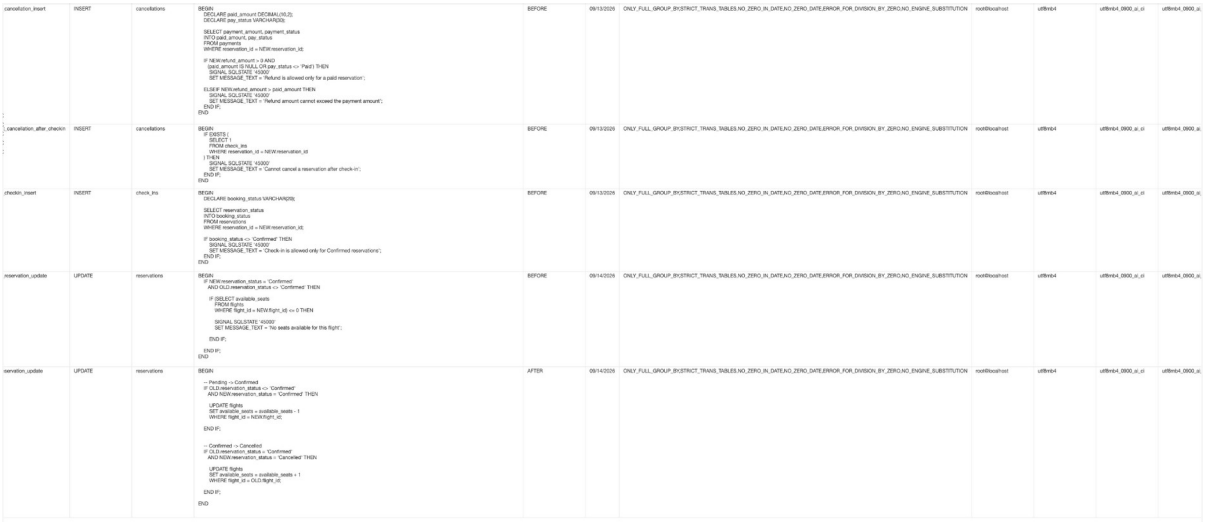
APPENDIX C — SCREENSHOTS AND TEST EVIDENCE

- Dashboard with database connection indicator and summary cards.
- Flight Search & Booking page with search criteria and results.
- Operations Center showing reservation workflow actions.
- Module page showing records and a CRUD operation.
- Reports page showing a database-derived report.
- Terminal or database output confirming schema and trigger definitions.
- Completed positive and negative test-case evidence.

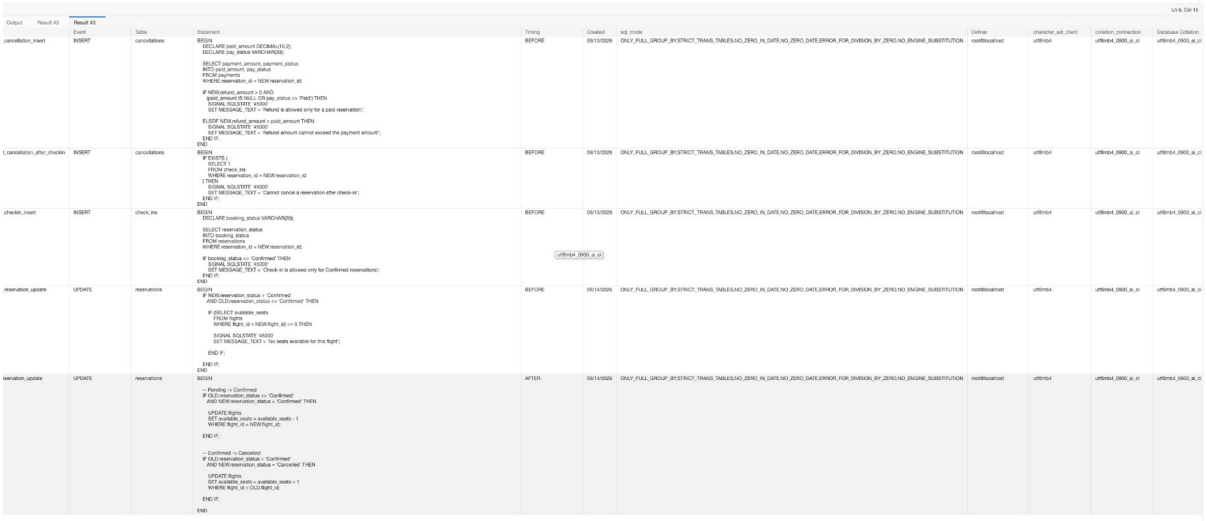
Appendix C contains screenshots and workflow evidence available at report preparation time. Tests not supported by captured evidence are explicitly marked not verified.

APPENDIX A: SELECTED SCREENSHOT EVIDENCE

The following screenshots provide a concise evidence set for database inspection, SQL rules, verification queries, and application operations. They are selected to avoid repetition and keep the report within the requested length.



Appendix Figure A1. Database objects and schema navigation in MySQL Workbench.



Appendix Figure A2. Database object verification and query evidence.

Output | Result #2 | Result #3 | Result #4 | Result #5 | Result #6 | Result #7 | Result #8 | Result #9 | Result #10

Table	Create Table
flights	CREATE TABLE `flights` (`flight_id` int NOT NULL AUTO_INCREMENT, `flight_number` varchar(20) NOT NULL, `airline` varchar(100) NOT NULL, `source` varchar(100) NOT NULL, `destination` varchar(100) NOT NULL, `departure_time` datetime NOT NULL, `arrival_time` datetime NOT NULL, `available_seats` int NOT NULL, PRIMARY KEY (`flight_id`), UNIQUE KEY `flight_number` (`flight_number`), CONSTRAINT `chk_available_seats` CHECK ((`available_seats` >= 0)), CONSTRAINT `chk_flight_times` CHECK ((`arrival_time` > `departure_time`))) ENGINE=InnoDB AUTO_INCREMENT=21 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

Appendix Figure A3. SQL trigger definition and business-rule implementation.

Output | Result #2 | Result #3 | Result #4 | Result #5 | Result #6 | Result #7 | Result #8 | Result #9 | Result #10

Table	Create Table
tickets	CREATE TABLE `tickets` (`ticket_id` int NOT NULL AUTO_INCREMENT, `reservation_id` int NOT NULL, `ticket_number` varchar(50) NOT NULL, `issue_date` datetime DEFAULT CURRENT_TIMESTAMP, `ticket_status` varchar(20) DEFAULT 'Issued', PRIMARY KEY (`ticket_id`), UNIQUE KEY `ticket_number` (`ticket_number`), KEY `reservation_id` (`reservation_id`), CONSTRAINT `tickets_ibfk_1` FOREIGN KEY (`reservation_id`) REFERENCES `reservations` (`reservation_id`) ON DELETE CASCADE) ENGINE=InnoDB AUTO_INCREMENT=11 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

Appendix Figure A4. SQL verification query output.

Output | Result #2 | Result #3 | Result #4 | Result #5 | Result #6 | Result #7 | Result #8 | **Result #9** | Result #10

Table	Create Table
baggage	CREATE TABLE `baggage` (`baggage_id` int NOT NULL AUTO_INCREMENT, `reservation_id` int NOT NULL, `baggage_weight` decimal(10,2) NOT NULL, `baggage_status` varchar(30) DEFAULT 'Checked-In', PRIMARY KEY (`baggage_id`), KEY `reservation_id` (`reservation_id`), CONSTRAINT `baggage_ibfk_1` FOREIGN KEY (`reservation_id`) REFERENCES `reservations` (`reservation_id`) ON DELETE CASCADE, CONSTRAINT `baggage_chk_1` CHECK ((`baggage_weight` >= 0)), CONSTRAINT `chk_baggage_weight` CHECK ((`baggage_weight` >= 0))) ENGINE=InnoDB AUTO_INCREMENT=9 DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_0900_ai_ci

Appendix Figure A5. Additional database validation evidence.

DATABASE MODULE

Reservations

Live records from the **reservations** table.

● MySQL

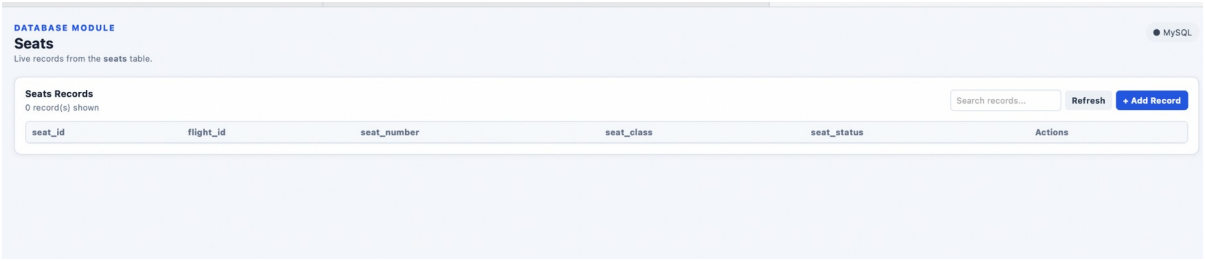
Reservations Records

11 record(s) shown — read-only (changed through the booking workflow)

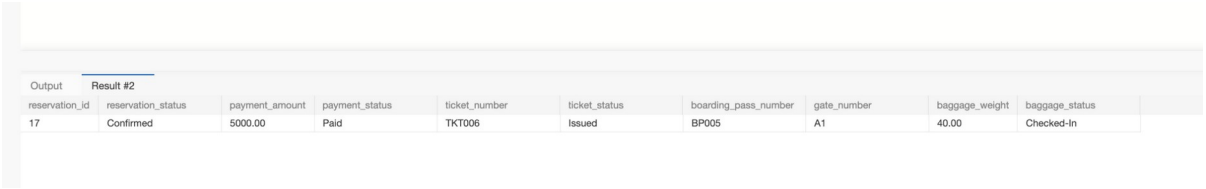
Search records... | Refresh

reservation_id	passenger_id	flight_id	booking_date	seat_number	reservation_status
16	22	5	2026-10-10	3D	Confirmed
15	10	5	2026-09-10	15A	Confirmed
14	9	4	2026-09-09	22B	Cancelled
13	8	3	2026-09-08	11C	Confirmed
12	7	2	2026-09-07	20A	Cancelled
11	6	1	2026-09-06	18B	Confirmed
5	5	5	2026-09-19	5D	Cancelled
4	4	4	2026-09-18	20A	Cancelled
3	3	3	2026-09-17	8C	Cancelled
2	2	2	2026-09-16	14B	Confirmed
1	1	1	2026-09-15	12A	Confirmed

Appendix Figure A6. Reservation and operational records in the application database.



Appendix Figure A7. Report/query result evidence.



Appendix Figure A8. Operations workflow status evidence.